



CS-250 : Lab 01

Pointers in C++

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Class: BESE 16B

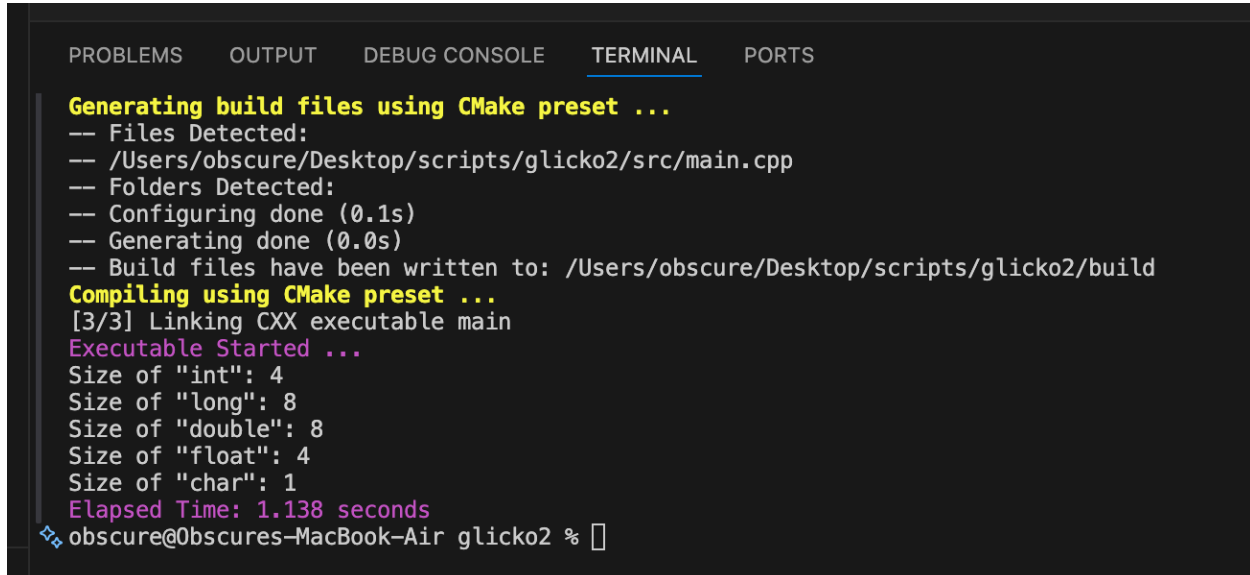
Batch: 2k25

Task 1: Memory occupied by int, long, double, float and char.

```
#include <iostream>

int main()
{
    // sizeof is used to print the size of the given type in bytes
    std::cout << "Size of \"int\": " << sizeof(int) << "\n";
    std::cout << "Size of \"long\": " << sizeof(long) << "\n";
    std::cout << "Size of \"double\": " << sizeof(double) << "\n";
    std::cout << "Size of \"float\": " << sizeof(float) << "\n";
    std::cout << "Size of \"char\": " << sizeof(char) << "\n";
}
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Generating build files using CMake preset ...
-- Files Detected:
-- /Users/obscure/Desktop/scripts/glicko2/src/main.cpp
-- Folders Detected:
-- Configuring done (0.1s)
-- Generating done (0.0s)
-- Build files have been written to: /Users/obscure/Desktop/scripts/glicko2/build
Compiling using CMake preset ...
[3/3] Linking CXX executable main
Executable Started ...
Size of "int": 4
Size of "long": 8
Size of "double": 8
Size of "float": 4
Size of "char": 1
Elapsed Time: 1.138 seconds
❖ obscure@Obscure-MacBook-Air glicko2 %
```

Task 2:

(a) Why does the memory address stored in pointer “pa” vary by 4?

Adding or removing a number from a pointer causes the address it is pointer to change by:

number of bytes in the pointer type (int in this case) × the number

It varies by 4 in this case because of int data type

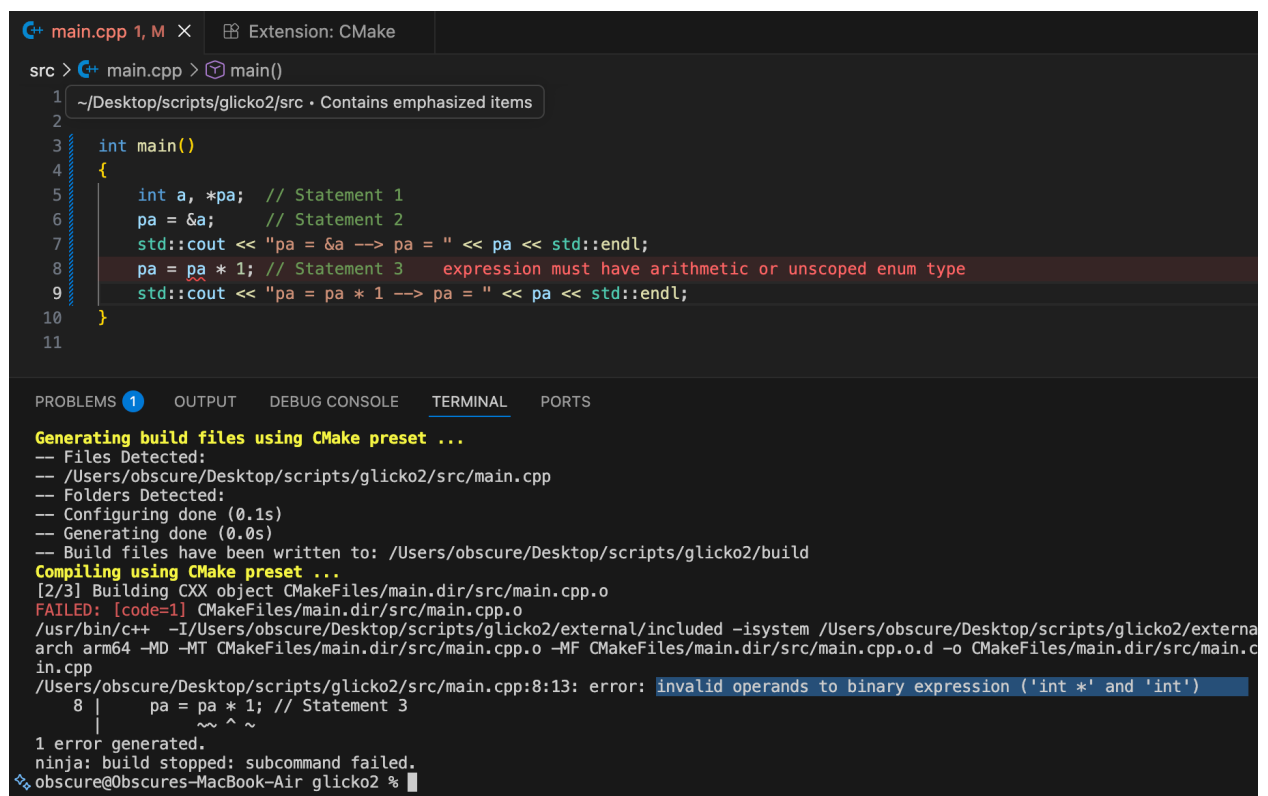
(b) Will the address still vary by 4 if the data type of the above-mentioned code changed from “int” to “long”? Explain your answer.

No, it will instead vary by 8* which is the size of long data type. So, “+ 1” would move it by 8 bytes forward and “- 3” would move it by $(8 * 3) = 24$ bytes backward.

[*8 on macOS or Linux, it is usually 4 bytes on Windows OS. \(Source given as hyperlink\)](#)

(c) If we try to multiply the address pointed to by “pa” what will happen? Is this logically or programmatically, correct? Attach screen shot of the output you get when you try this multiplication.

This is programmatically incorrect and gives a syntax error.



```
src > main.cpp > main()
1 ~/Desktop/scripts/glicko2/src - Contains emphasized items
2
3 int main()
4 {
5     int a, *pa; // Statement 1
6     pa = &a;    // Statement 2
7     std::cout << "pa = &a --> pa = " << pa << std::endl;
8     pa = pa * 1; // Statement 3 expression must have arithmetic or unscoped enum type
9     std::cout << "pa = pa * 1 --> pa = " << pa << std::endl;
10 }
11

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Generating build files using CMake preset ...
-- Files Detected:
-- /Users/obscure/Desktop/scripts/glicko2/src/main.cpp
-- Folders Detected:
-- Configuring done (0.1s)
-- Generating done (0.0s)
-- Build files have been written to: /Users/obscure/Desktop/scripts/glicko2/build
Compiling using CMake preset ...
[2/3] Building CXX object CMakeFiles/main.dir/src/main.cpp.o
FAILED: [code=1] CMakeFiles/main.dir/src/main.cpp.o
/usr/bin/c++ -I/Users/obscure/Desktop/scripts/glicko2/external/included -isystem /Users/obscure/Desktop/scripts/glicko2/external/included -MD -MT CMakeFiles/main.dir/src/main.cpp.o -MF CMakeFiles/main.dir/src/main.cpp.o.d -o CMakeFiles/main.dir/src/main.cpp.o
/Users/obscure/Desktop/scripts/glicko2/src/main.cpp:8:13: error: invalid operands to binary expression ('int *' and 'int')
      8 |     pa = pa * 1; // Statement 3
        |           ^~ ^~
1 error generated.
ninja: build stopped: subcommand failed.
obscure@Obscure-MacBook-Air glicko2 %
```

Task 3: Array Access using pointer to the array

```
#include <iostream>

int main()
{
    int list[5] = {3, 6, 9, 12, 15};
    int *pArr = list;

    for (int i = 0; i < 5; i++)
    {
        // elements of array are stored sequentially in memory,
        // one after the other; using pointer arithmetic we
        // can get the next element, given we have the type of
        // the elements in the array as well, "int" in this case
        std::cout << "Element " << i << ": " << *(pArr + i) << "\n";
    }
}
```

Output:

```
Generating build files using CMake preset ...
-- Files Detected:
-- /Users/obscure/Desktop/scripts/glicko2/src/main.cpp
-- Folders Detected:
-- Configuring done (0.1s)
-- Generating done (0.0s)
-- Build files have been written to: /Users/obscure/Desktop/scripts/glicko2/build
Compiling using CMake preset ...
[3/3] Linking CXX executable main
Executable Started ...
Element 0: 3
Element 1: 6
Element 2: 9
Element 3: 12
Element 4: 15
Elapsed Time: 1.103 seconds
❖ obscure@Obscure-MacBook-Air glicko2 %
```

Task 4: Output of C++ snippets

Output for Code A)

1
4
7
9

Explanation:

```
p = p + 2;
```

In this line of code, we are moving the pointer ahead in the memory by $(4 \times 2) = 8$ bytes storing the address of “9” inside of p.

Output for Code B)

4 4 5 7

Explanation:

In this code, we first initialize a variable (a), 2 pointers (p, q), and an array (arr).

p is pointed to arr, and q is pointed to p

The value at the first element is set to 0 using the pointer.

In a for loop, we traverse through the array using pointer p, and insert elements into it.

We print those elements using pointer q in another for loop.

Task 5:

- (a) Write code of a function that swaps values of variables a and b. Input to the function should be the address of both the variables.

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

- (b) Write code of a function that swaps values of the variables a and b using pointer-to-pointer variables ppa and ppb.

```
void swap(int **ppa, int **ppb)
{
    int temp = **ppa;
    **ppa = **ppb;
    **ppb = temp;
}
```

Task 6:

```
#include <iostream>

int main()
{
    int x;
    double y;
    void *ptr;

    std::cout << "Enter an integer: ";
    ptr = &x;
    std::cin >> *((int *)ptr);
    std::cout << "You entered integer: " << *((int *)ptr) << std::endl;
    // *((int *)ptr) = (ptr casted to integer pointer) dereferenced wholly

    std::cout << "Enter a double: ";
    ptr = &y;
    std::cin >> *((double *)ptr);
    std::cout << "You entered double: " << *((double *)ptr) << std::endl;
    // *((double *)ptr) = (ptr casted to double pointer) dereferenced wholly

    return 0;
}
```